

Dynamic Graph Representation Learning

Prof. O-Joun Lee

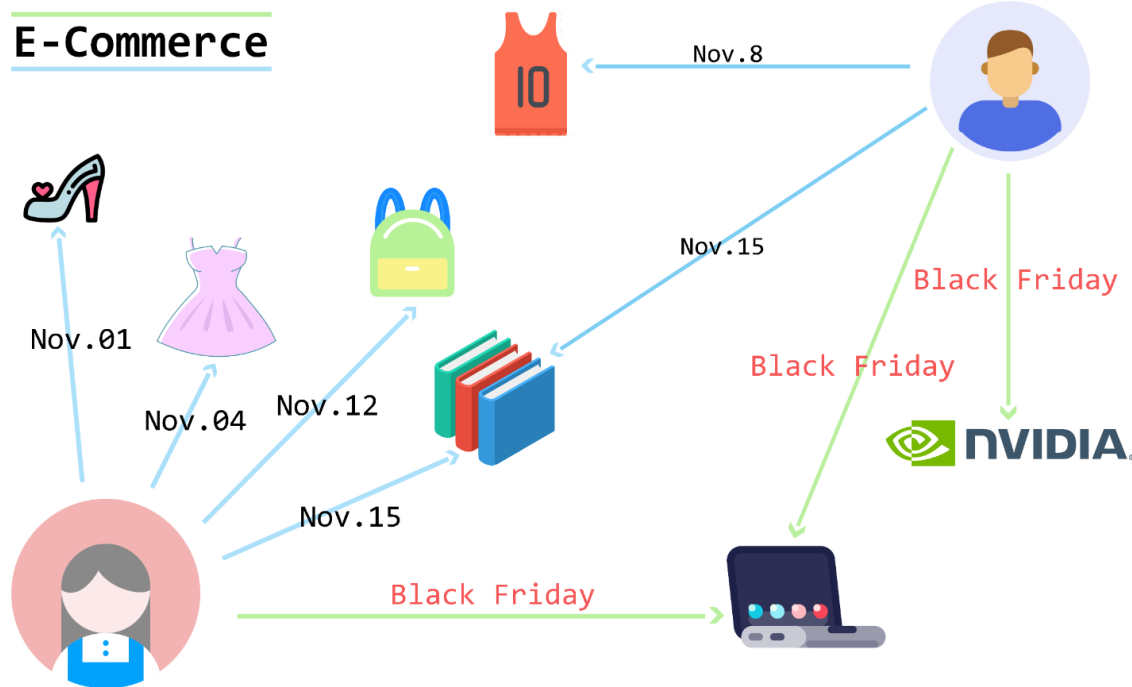
Dept. of Artificial Intelligence,
The Catholic University of Korea
ojlee@catholic.ac.kr

Contents

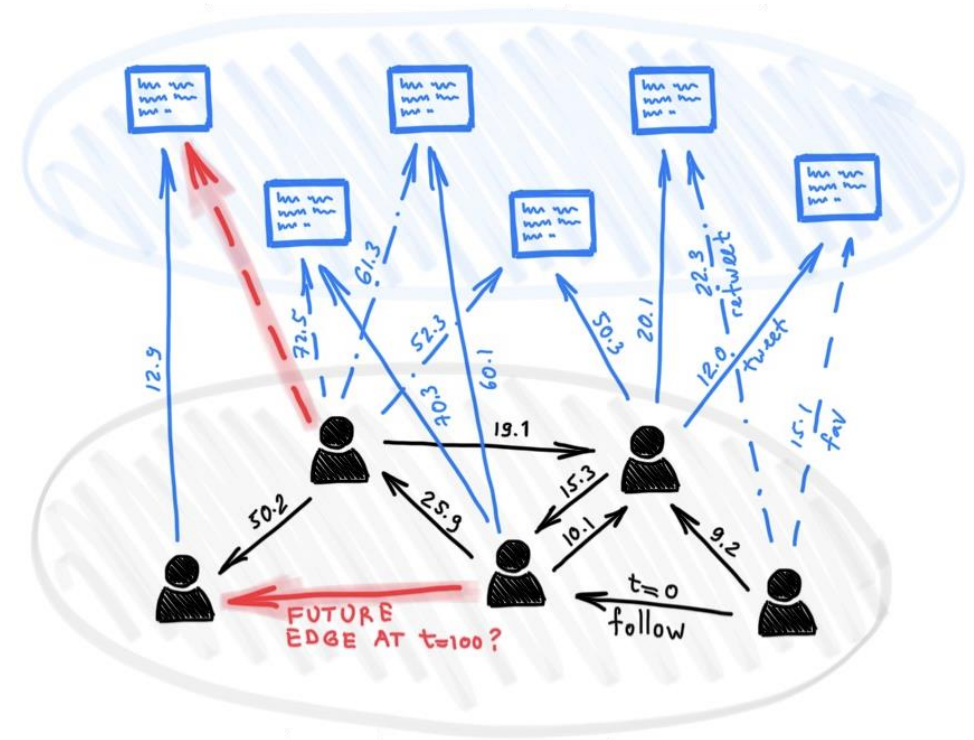


- Overview of Dynamic Graph
- Matrix factorization-based:
 - TIMERS
- SkipGram-based
 - dynnode2vec: Scalable Dynamic Network Embedding
 - Continuous-Time Dynamic Network Embeddings
- Autoencoder-based
 - DynGEM
- Others
 - DynamicTriad

- Previous random walk methods – DeepWalk, Node2Vec, LINE, Struc2Vec, Metapath2Vec, etc – are relied on static graph
- However, the networks in the real world are dynamic
 - Evolving over time



An illustration of user-item graph



An illustration of social graph

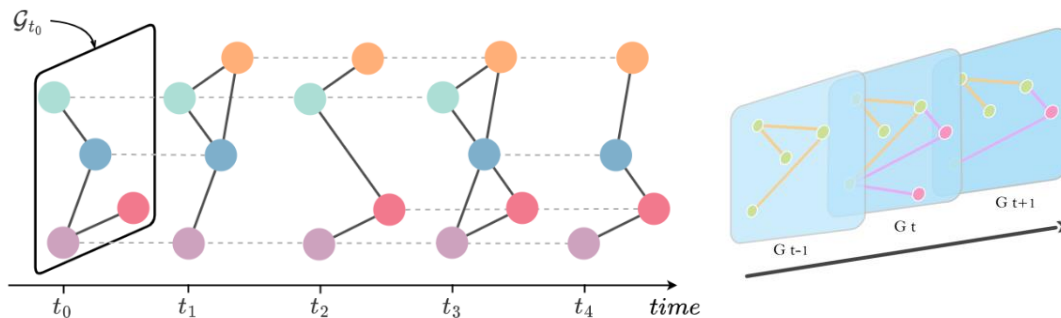
Feature	Static Graph	Dynamic Graph
Topology	Fixed	Changes over time
Nodes & edges	Constant	Nodes may join/leave; edges may form/dissolve
Attributes	Usually fixed	May evolve (weights, labels, etc.)
Representation	Single snapshot	Sequence of snapshots or event streams
Temporal reasoning	Not required	Essential for evolution, causality, temporal patterns
Algorithmic complexity	Generally simpler	Higher due to time dimension
Storage & computation	Simpler to store, visualize, compute	Must support updates, streaming, or event logs

- **Problem:** Learning dynamic node representations
- **Challenges:**
 - Time-varying graph structures: Links and node can emerge and disappear; communities are changing all the time
 - Require the node representations capture both **structural proximity** (as in static cases) and their **temporal evolution**
 - Time intervals of events are uneven
 - Causes of the change: can come from different aspects, e.g.in co-authorship network, research community & career stage perspectives
 - Require modeling multi-faceted variations

Discrete-Time Dynamic Graphs (D-TDG)

Discrete model: Sequence of network snapshots within a given time interval

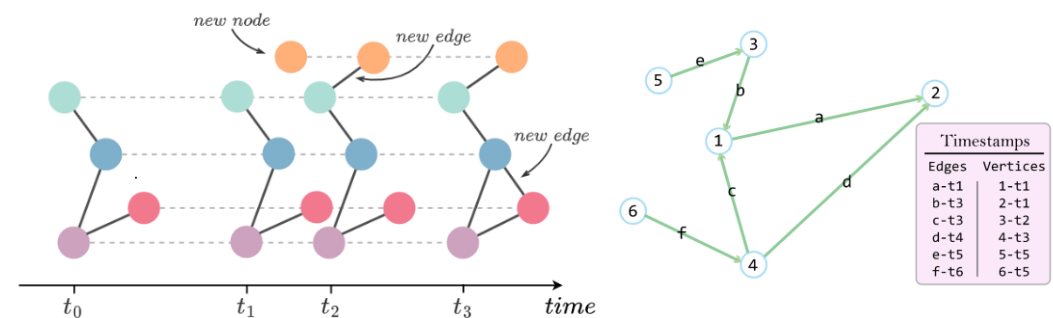
$G = \{G_1, \dots, G_T\}$, where T is the number of snapshots. Each snapshot $G_t = (V_t, E_t)$ is a static network recorded at time t



Continuous-Time Dynamic Graphs (C-TDG)

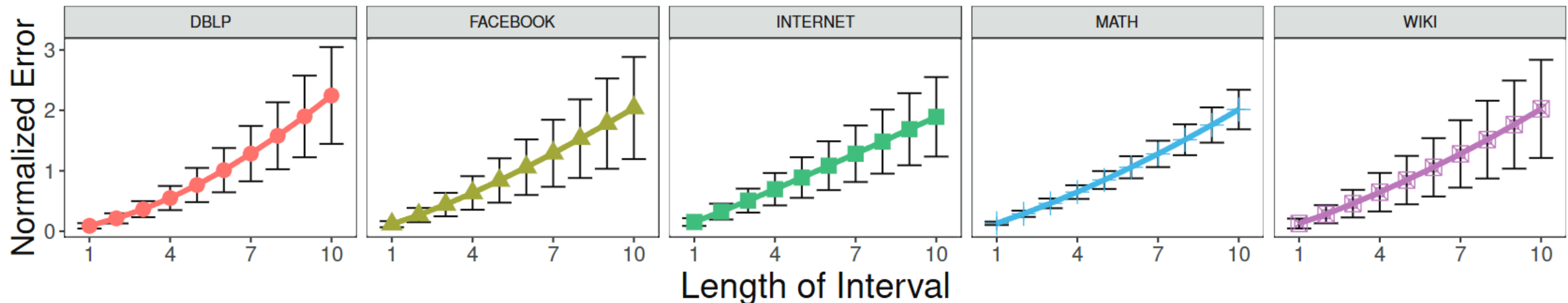
Continuous model: a network with edges and nodes annotated with timestamps

We have $G = (V_T, E_T, \mathcal{T})$ where $\mathcal{T}: V, E \rightarrow \mathbb{R}^+$ is a function that maps each edge and node to a corresponding timestamp



- **Node addition/deletion**
e.g. New users join the platform / Users leave the platform
- **Feature update:** Graph structure remains unchanged while attributes evolve.
e.g. User profile change: Age increases / Country of residence changes
- **Edge addition/deletion**
e.g. Follow/unfollow on social media
- **Edge weight updates**
e.g. Interaction frequency, Number of exchanged messages
- **Relation updates:** Edges may carry semantic labels. A relation update can be viewed as:
 1. Remove old edge (u, r_1, v)
 2. Add new edge (u, r_2, v)e.g $(u, \text{friend}, v) \rightarrow (u, \text{engaged}, v)$

- **Motivation:** Singular Value Decomposition (SVD) on Dynamic Networks
 - SVD is foundational for network tasks: link prediction, spectral analysis, parameter characterization
 - Real-world networks are **dynamic** - nodes/edges appear and disappear continuously
 - **Incremental SVD** avoids full recomputation, but introduces **approximation error** at each step
 - Error accumulates over time → SVD must be restarted



➤ Key Notations and Definitions

- **Dynamic Network:** Represented as a sequence of adjacency matrices $A_0, A_1, \dots, A_T$ where T is the number of time slices.
- **Adjacency Matrix (A_t):** The state of the network at time t .
- **Dynamic Change (ΔA_t):** Defined as $\Delta A_t = A_t - A_{t-1}$, representing edge additions or deletions.

➤ SVD Reconstruction Loss

- The objective of Singular Value Decomposition (SVD) for networks is to minimize the reconstruction loss J :

$$J = \|S - U \cdot \Sigma \cdot V^T\|_F^2$$

Where:

- S : Symmetric similarity matrix derived from the adjacency matrix (e.g., $S = S(A)$).
- $U, V \in \mathbb{R}^{N \times k}$: Unitary matrices representing node embeddings.
- $\Sigma \in \mathbb{R}^{k \times k}$: Non-negative diagonal matrix containing the top- k singular values.
- k : Target dimensionality of the low-rank space.

➤ Minimum Model Loss

- The intrinsic minimum loss $L(S, k)$ of the SVD model (which cannot be reduced by restarting) is:

$$L(S, k) = \sum_{l=k+1}^N \lambda_l^2$$

Where:

- λ_l : Eigenvalues of S in descending order of magnitude.
- N : Total number of nodes.
- The loss is the sum of squares of the discarded eigenvalues beyond the top k .

➤ Incremental SVD Update Function $F(\cdot)$

- The function $F(\cdot)$ represents the **incremental update mechanism** that integrates new network changes without performing a full SVD restart.
- **The Update Equation**

$$[U_t, \Sigma_t, V_t] = F([U_{t-1}, \Sigma_{t-1}, V_{t-1}], S_{t-1}, \Delta S_t)$$

Where:

- $[U_{t-1}, \Sigma_{t-1}, V_{t-1}]$: The **previous SVD results** (singular vectors and values) from time slice $t - 1$
- S_{t-1} : The **similarity matrix** from the previous time slice.
- ΔS_t : The **change in similarity** at the current time slice (e.g., new edges or weight shifts).
- $[U_t, \Sigma_t, V_t]$: The **approximated results** for the current time slice.

➤ The Optimization Goal: Maximum-Error-Bounded Restart

- The goal is to minimize the total number of restarts while keeping the accumulated error below a threshold Θ :

$$\min \sum_{t=1}^T c_t \text{ subject to } G \leq \Theta$$

- $c_t \in \{0,1\}$: Binary indicator where 1 triggers an SVD restart at time t , and 0 uses incremental updates.
- G : The maximum relative error caused by incremental approximations, defined as:

$$G = \max_{1 \leq t \leq T} \frac{J(t) - L(S_t, k)}{L(S_t, k)}$$

- $J(t)$: Actual reconstruction loss at time t using the current (potentially stale) SVD results.
- $L(S_t, k)$: Theoretical minimum loss if SVD were restarted at time t .

The Framework for Error Monitoring

Challenge: Computing $L(S_t, k)$ exactly requires a full SVD same cost as a restart itself.

Efficiency via Lower Bounding

TIMERS circumvents this by using a theoretical lower bound $B(t)$:

$$L(S_t, k) \geq B(t)$$

By substituting $L(S_t, k)$ with $B(t)$, the monitoring condition becomes a sufficient condition for the error bound:

$$\frac{J(t) - B(t)}{B(t)} \leq \Theta \rightarrow \frac{J(t) - L(S_t, k)}{L(S_t, k)} \leq \Theta$$

→ Benefit: $B(t)$ is calculated in linear time relative to local dynamic changes, making real-time monitoring feasible

A Lower Bound of SVD Minimum Loss

To avoid the high cost of calculating the exact SVD minimum loss at every time step, TIMERS employs a lower bound derived from matrix perturbation theory (Stewart 1990).

The Lower Bound Theorem

If the similarity matrix S and its change ΔS are symmetric, the new minimum loss is bounded as follows:

$$L(S + \Delta S, k) \geq L(S, k) + \Delta \text{tr}^2(S + \Delta S, S) - \sum_{l=1}^k \lambda_l$$

where $\lambda_1 \geq \lambda_2 \dots \geq \lambda_k$ are the top- k eigenvalues of $\nabla_{S^2} = \mathbf{S} \cdot \Delta \mathbf{S} + \Delta \mathbf{S} \cdot \mathbf{S} + \Delta \mathbf{S} \cdot \Delta \mathbf{S}$, and

$$\Delta \text{tr}^2(\mathbf{S} + \Delta \mathbf{S}, \mathbf{S}) = \text{tr}((\mathbf{S} + \Delta \mathbf{S}) \cdot (\mathbf{S} + \Delta \mathbf{S})) - \text{tr}(\mathbf{S} \cdot \mathbf{S}).$$

Using the Bound in Monitoring

Setting the bound $B(t)$

The bound is defined by integrating the results from the last SVD restart with the accumulated structural changes:

$$B(t) = L(S_{t'}, k) + \Delta \text{tr}^2(S_t, S_{t'}) - \Lambda_k(\nabla S^2)$$

- $L(S_{t'}, k)$: The **baseline minimum loss** calculated during the most recent full SVD restart at time t' .
- $\Delta \text{tr}^2(S_t, S_{t'})$: The **change in total matrix energy** between the current time t and the restart time t' .
- $\Lambda_k(\nabla S^2)$: The **sum of the top- k eigenvalues** of the perturbation matrix ∇S^2 .

Update Logic: If the monitoring condition $\frac{J(t) - B(t)}{B(t)} > \Theta$ is met, a new SVD is triggered, and t' is updated to the current t , resetting the baseline for the bound.

Complexity Analysis

The bound $B(t)$ is highly efficient because it focuses only on local changes rather than the entire network:

Complexity:

$$O(M_S + M_L k + N_L k^2)$$

- M_S : Number of changed elements in the similarity matrix.
- M_L, N_L : Number of non-zero elements/rows in ∇S^2 .
- **Linearity**: For most real-world networks (e.g., preferential attachment), the complexity is linear with respect to the number of local dynamic changes

Algorithm 1 TIMERS: Theoretically Instructed Maximum Error-bounded Restart of SVD

Require: Static Adjacency Matrix $\mathbf{A}_0$, Dynamic Changes $\Delta\mathbf{A}_1 \dots \Delta\mathbf{A}_T$, Similarity Function $\mathcal{S}(\cdot)$, Dimensionality k , Error Threshold Θ , Incremental SVD method $\mathcal{F}(\cdot)$

Ensure: SVD results in each time slice $[\mathbf{U}_t, \mathbf{\Sigma}_t, \mathbf{V}_t]$

```

1: Calculate the initial similarity  $\mathbf{S}_0 = \mathcal{S}(\mathbf{A}_0)$ 
2: Calculate SVD on  $\mathbf{S}_0$  to obtain  $[\mathbf{U}_0, \mathbf{\Sigma}_0, \mathbf{V}_0]$ 
3: for  $t$  in  $1:T$  do
4:   Calculate the similarity change in that time slice  $\Delta\mathbf{S}_t$ 
5:   Use  $\mathcal{F}(\cdot)$  to get updated SVD results  $[\mathbf{U}_t, \mathbf{\Sigma}_t, \mathbf{V}_t]$ 
6:   Compute loss  $\mathcal{J}(t)$  and bound  $B(t)$  using (13)
7:   if  $\frac{\mathcal{J}(t) - B(t)}{B(t)} > \Theta$  then
8:     Calculate SVD on  $\mathbf{S}_t$  to obtain new  $[\mathbf{U}_t, \mathbf{\Sigma}_t, \mathbf{V}_t]$ 
9:   end if
10:  Return  $[\mathbf{U}_t, \mathbf{\Sigma}_t, \mathbf{V}_t]$ 
11: end for
    
```

- Agnostic to the choice of incremental SVD method $\mathcal{F}(\cdot)$
- General across weighted, unweighted, signed, unsigned networks

➤ Definitions

- Weighted graph: $G(V, E)$ with adjacency matrix S , where $s_{ij} > 0$ iff $(i, j) \in E$
- **Graph embedding**: mapping $f: V \rightarrow \mathbb{R}^d$, $d \ll |V|$, preserving proximity
- **Dynamic graph**: sequence of snapshots $\mathcal{G} = \{G_1, \dots, G_T\}$
- **Growing graph setting**: $V_t \subseteq V_{t+1}$ (nodes only join, not leave)
 → If a node is **removed**, the paper treats it as still being part of the graph but with **zero weights** on all its edges to other nodes.
- **Dynamic embedding**: time-series of mappings $\mathcal{F} = \{f_1, \dots, f_T\}$

➤ Stability Metrics

Measuring Embedding Stability

Absolute Stability:

$$S_{\text{abs}}(\mathcal{F}; t) = \frac{|F_{t+1}(V_t) - F_t(V_t)|_F}{|S_{t+1}(V_t) - S_t(V_t)|_F}$$

$F_t(V_t) \in \mathbb{R}^{|V_t| \times d}$: The embedding matrix of all nodes in the set V_t at time t .

Relative Stability (scale-invariant):

$$S_{\text{rel}}(\mathcal{F}; t) = \frac{|F_{t+1}(V_t) - F_t(V_t)|_F / |F_t(V_t)|_F}{|S_{t+1}(V_t) - S_t(V_t)|_F / |S_t(V_t)|_F}$$

$S_t(V_t)$: The weighted adjacency matrix of the induced subgraph of node set V_t at time t .

Stability Constant:

$$K_S(\mathcal{F}) = \max_{\tau, \tau'} |S_{\text{rel}}(\mathcal{F}; \tau) - S_{\text{rel}}(\mathcal{F}; \tau')|$$

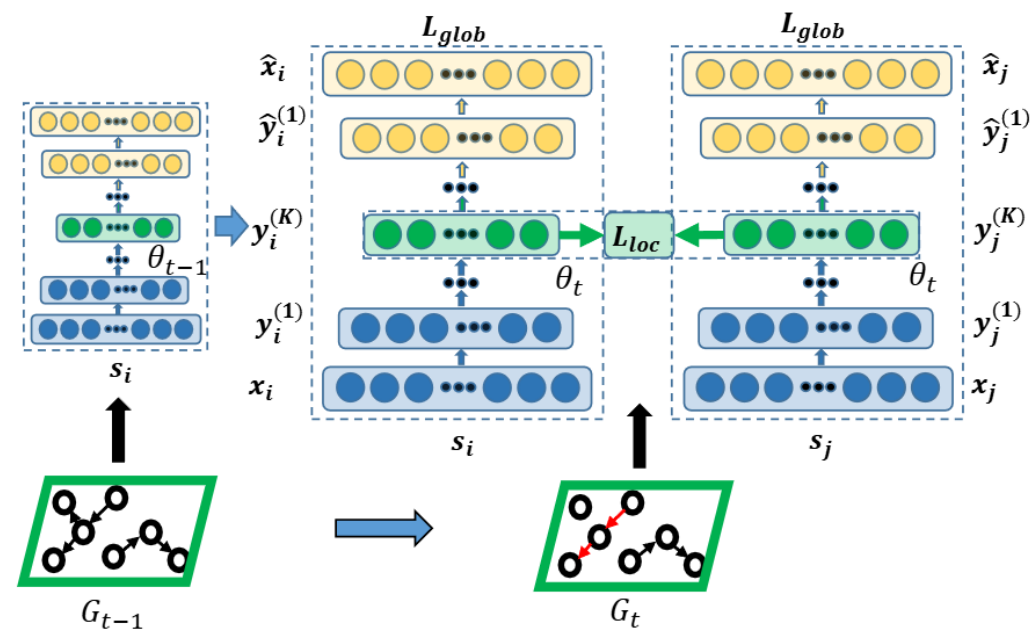
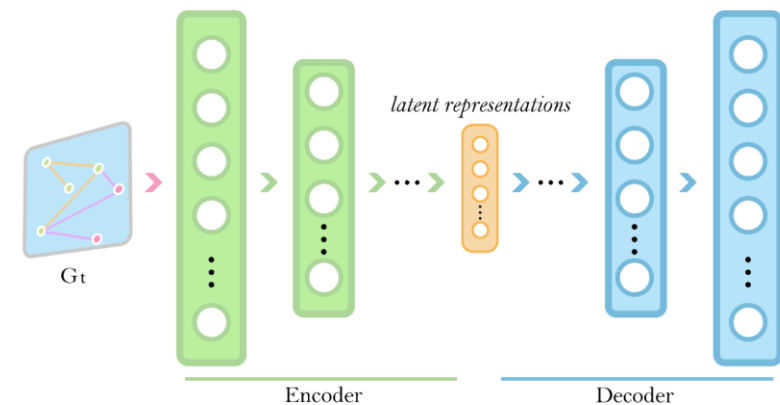
The maximum difference between relative stability values across any two time steps τ and τ' .

Smaller $K_S(\mathcal{F}) \Rightarrow$ more stable embedding over time

➤ Model Architecture

Deep Autoencoder for Graph Embedding

- Input to node v_i : its neighborhood vector $x_i = s_i \in \mathbb{R}^m$
- Encoder: $x_i \xrightarrow{K \text{ layers}} y_i^{(K)} \in \mathbb{R}^d$ (embedding)
- Decoder: $y_i^{(K)} \rightarrow \hat{x}_i$ (neighborhood reconstruction)
- Weights $\theta = \{W^{(k)}, \widehat{W^{(k)}}, b^{(k)}, \widehat{b^{(k)}}\}_{k=1}^K$
- **Key design choice:** ReLU activations
 - Supports positive-valued weighted graphs
 - Avoids vanishing gradients
 - Faster derivative computation



➤ Loss Function

Multi-Objective Optimization

$$\mathcal{L}_{\text{net}} = \mathcal{L}_{\text{glob}} + \alpha \mathcal{L}_{\text{loc}} + \nu_1 \mathcal{L}_1 + \nu_2 \mathcal{L}_2$$

Term	Formula	Purpose
$\mathcal{L}_{\text{glob}}$	$ (\hat{X} - X) \odot B _F^2$	2nd-order proximity (global neighborhood)
$\mathcal{L}_{\text{loc}}$	$\sum_{i,j} s_{ij} y_i - y_j _2^2$	1st-order proximity (local structure)
$\mathcal{L}_1$	$\sum_k (W^{(k)} _1 + \hat{W}^{(k)} _1)$	Sparsity regularization
$\mathcal{L}_2$	$\sum_k (W^{(k)} _F^2 + \hat{W}^{(k)} _F^2)$	Overfitting prevention

➤ Handling Growing Graphs: PropSize

To handle growing graph sizes, DynGEM utilizes the **PropSize** heuristic to dynamically adjust the neural network architecture. When $|V_t| > |V_{t-1}|$, the network must grow. PropSize determines new layer sizes:

For each consecutive encoder layer pair (l_k, l_{k+1}) :

$$\text{size}(l_{k+1}) \geq \rho \times \text{size}(l_k), \quad 0 < \rho < 1$$

where $0 < \rho < 1$ is a hyperparameter.

- If violated: **widen** l_{k+1} via *Net2WiderNet* (Chen et al., 2015)
- If violated at **penultimate-embedding** pair: deepen via *Net2DeeperNet* (Chen et al., 2015)

Note: Embedding layer size d is always fixed

➤ Stability by reusing previous step embedding

Incremental Training Strategy

$$\theta_t \leftarrow \theta_{t-1} \quad (\text{before training on } G_t)$$

- G_1 : train from random initialization
- $G_t, t \geq 2$: initialize from θ_{t-1} , then fine-tune
- **Key insight**: model only needs to learn the delta $G_t \setminus G_{t-1}$
- Convergence is fast for $t \geq 2$ - few gradient steps needed
- No explicit temporal regularizer

Expected speedup for T snapshots:

$$\text{Speedup} = \frac{T \cdot n_s}{n_s + (T - 1) \cdot n_i}$$

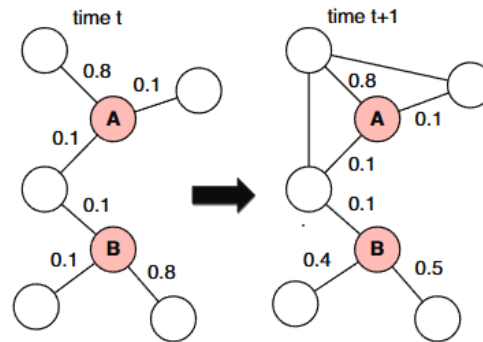
where n_s = scratch iterations, n_i = incremental iterations ($n_i \ll n_s$)

➤ DynGEM algorithm

Algorithm 1: Algorithm: DynGEM

- 1: **Input:** Graphs $G_t = (V_t, E_t)$, $G_{t-1} = (V_{t-1}, E_{t-1})$, Embedding Y_{t-1} , autoencoder weights θ_{t-1}
 - 2: **Output:** Embedding Y_t
 - 3: From G_t , generate adjacency matrix S , penalty matrix B
 - 4: Create the autoencoder model with initial architecture
 - 5: Initialize θ_t randomly **if** $t = 1$, **else** $\theta_t = \theta_{t-1}$
 - 6: **if** $|V_t| > |V_{t-1}|$ **then**
 - 7: Compute new layer sizes with PropSize heuristic
 - 8: Expand autoencoder layers and/or insert new layers
 - 9: **end if**
 - 10: Create set $\mathcal{S} = \{(s_i, s_j)\}$ for each $e = (v_i, v_j) \in E_t$
 - 11: **for** $i = 1, 2, \dots$ **do**
 - 12: Sample a minibatch M from $\mathcal{S}$
 - 13: Compute gradients $\nabla_{\theta_t} L_{net}$ of objective L_{net} on M
 - 14: Do gradient update on θ_t with nesterov momentum
 - 15: **end for**
-

- Running example: Two vertices can have **identical**
 - At time t : User A and B both have 3 friends, similar tie-strength distributions $\rightarrow$ static methods embed them similarly
 - At time $t + 1$: A 's friends interconnect; B 's do not $\rightarrow$ fundamentally different social strategies



- **Key requirement:** Embeddings must capture evolutionary structure, not just instantaneous structure

➤ Dynamic Network

A series of undirected weighted graph snapshots $\{G_1, \dots, G_T\}$, where:

$$G_t = (V, E_t, W_t), \quad 1 \leq t \leq T$$

Each edge $e_{ij}^t \in E_t$ has weight $w_{ij}^t \in \mathbb{R}^+$ (e.g., call frequency, co-authorship count)

➤ Dynamic Network Embedding

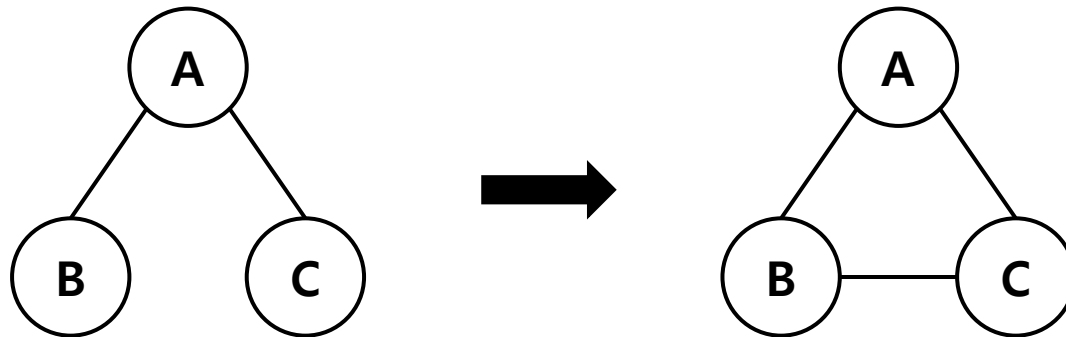
Learn a mapping $f^t: v_i \mapsto \mathbb{R}^d$ for each time step t , such that $\mathbf{u}_i^t := f^t(v_i)$ preserves:

1. Structural similarity at time t
2. Tendency of vertices to develop future relationships

Core Concept: Triadic Closure

The triadic closure process is a fundamental mechanism in the formation and evolution of dynamic networks. It describes how relationships develop among groups of three vertices.

- **Open Triad:** A group of three vertices where only two pairs are connected (e.g., v_i and v_j are both friends with v_k , but not with each other).
- **Closed Triad:** A group where all three vertices are connected to each other.
- **The Process:** Modeling how an open triad evolves into a closed triad over time.



Mathematical Modeling

The model assumes a vertex v_k decides whether to introduce its friends v_i and v_j based on their distance in the latent space, represented by the vector x_{ijk}^t :

$$x_{ijk}^t = w_{ik}^t \times (u_k^t - u_i^t) + w_{jk}^t \times (u_k^t - u_j^t)$$

- w_{ik}^t : Tie strength between v_i and v_k at time t .
- u_i^t : Embedding vector of vertex i at time t .

Probability of Closure

The probability that an open triad (v_i, v_j, v_k) becomes closed at time $t + 1$ is modeled as:

$$P_{tr}^t(i, j, k) = \frac{1}{1 + \exp(-\langle \theta, x_{ijk}^t \rangle)}$$

- θ : A parameter vector capturing social strategy.

Probability of Link Creation

A pair of unlinked vertices v_i and v_j often shares multiple common neighbors $B^t(i, j)$. The model assumes that a new link e_{ij} is created if at least one common friend successfully introduces them.

Probability of Link Formation at $t + 1$:

$$P_{tr+}^t(i, j) = \sum_{\alpha_{t,i,j} \neq 0} \prod_{k \in B^t(i, j)} (P_{tr}^t(i, j, k))^{\alpha_{t,i,j}^k} \times (1 - P_{tr}^t(i, j, k))^{(1 - \alpha_{t,i,j}^k)}$$

- $\alpha_{t,i,j}^k = 1$: If the open triad (v_i, v_j, v_k) develops into a closed triad.
- The sum captures all scenarios where at least one friend v_k bridges the connection.

Probability of No Link

If v_i and v_j are not influenced by any of their common friends, the open triads remain open, and no edge is formed at the next time step.

Probability of Edge Not Being Created:

$$P_{tr-}^t(i, j) = \prod_{k \in B^t(i, j)} (1 - P_{tr}^t(i, j, k))$$

- This represents the joint probability that every common neighbor v_k fails to close the triad.
- This formula allows the model to learn from "negative" instances (pairs that stay unlinked).

Loss Functions

Three components in the DynamicTriad objective:

1. Triadic closure loss:

$$\mathcal{L}_{\text{tr}}^t = - \sum_{(i,j) \in S_+^t} \log P_{\text{tr}+}^t(i,j) - \sum_{(i,j) \in S_-^t} \log P_{\text{tr}-}^t(i,j)$$

where S_+^t = links created at $t + 1$, S_-^t = links not created at $t + 1$

2. Social homophily loss:

$$L_{\text{sh}}^t = \sum_{\substack{(j,k) \in E_+^t \\ (j',k') \in E_-^t}} h(w_{jk}, [g^t(j,k) - g^t(j',k') + \xi]_+)$$

Connected pairs should be closer in embedding space than disconnected pairs

3. Temporal smoothness loss:

$$\mathcal{L}_{\text{smooth}}^t = \sum_{i=1}^N |u_i^t - u_i^{t-1}|_2^2$$

Network evolution is assumed locally smooth in the latent space

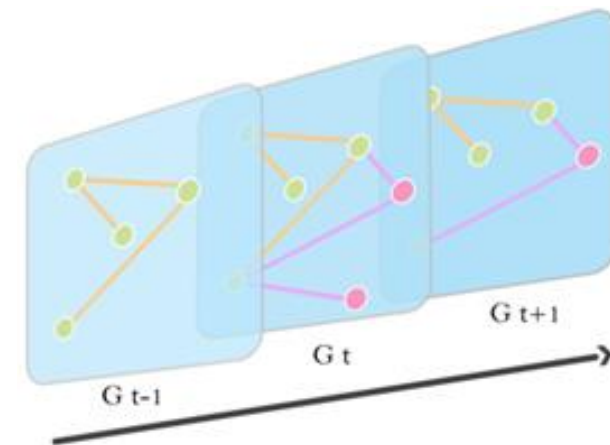
Full Objective

$$\arg \min_{\{\mathbf{u}_i^t\}, \boldsymbol{\theta}} \sum_{t=1}^T (\mathcal{L}_{\text{sh}}^t + \beta_0 \mathcal{L}_{\text{tr}}^t + \beta_1 \mathcal{L}_{\text{smooth}}^t)$$

Hyperparameter	Role
β_0	Weight of triadic closure dynamics
β_1	Weight of temporal smoothness regularization

- $\boldsymbol{\theta}$: shared social strategy vector - learned globally across all triads
- $\mathbf{u}_i^t$: the low-dimensional representation (a vector of length d) of a specific vertex i at a specific time step t .

- **Motivation:** Networks in the real world are always evolving
 - New users (new nodes) in social networks, new citations (new edges) in citation networks
 - Users may delete friends (delete edges) or some users may leave the network (delete nodes)
- The static graph embedding methods are not capable of dealing with the critical challenge involved in dynamic networks
 - **Disadvantage:** embedding vectors for each timestamp are in different spaces
 - Lead to learn embedding vectors separately is a time-consuming process
- **Solution:** dynnode2vec method to modify the node2vec method
 - Employ the previous learned embedding vectors as initials weights for the skip-gram model



- Given a dynamic graph as a sequence $G_1, G_2, \dots, G_T$ from timestamp 1 to T
- Each graph at time t is defined as $G_t = (V_t, E_t)$
- **Evolving Random Walk Generation:**
 - Only generates random walks for the set of "evolving nodes" (ΔV_t) that have changed between consecutive timestamps t and $t + 1$:

$$\Delta V_t = V_{add} \cup \{v_i \in V_t | \exists e_i = (v_i, v_j) \in (E_{add} \cup E_{del})\}$$

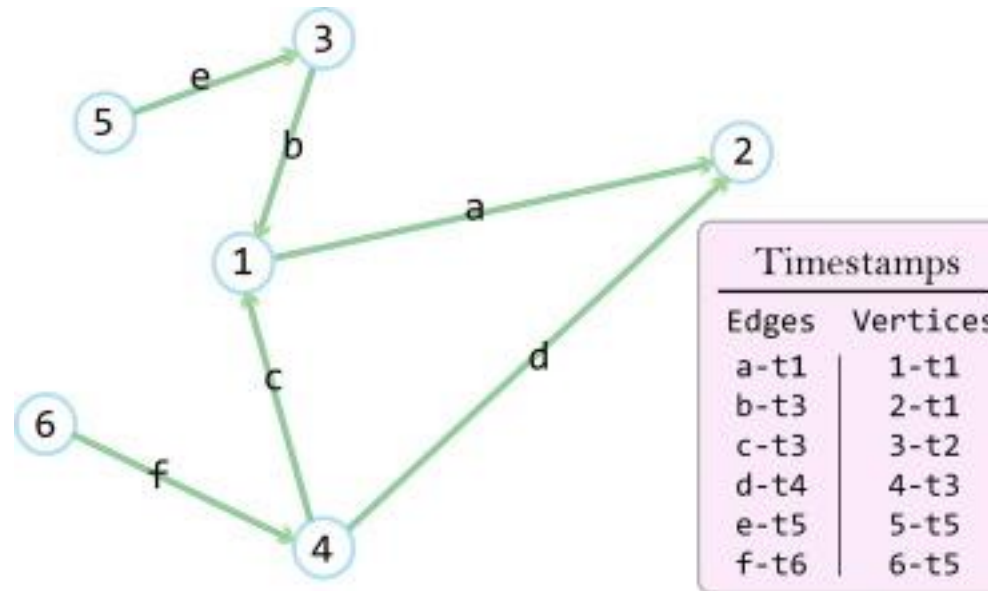
- Change: New nodes added, existing nodes deleted, or edges added/removed for existing nodes)
- **Dynamic Skip-gram Model:**
 - Initialize the skip-gram model at timestamp t with the pre-trained embedding vectors from the previous timestamp $t - 1$.
 - Vocabulary is updated based on the new evolving random walks, and Skip-gram t is retrained using only the new evolving random walks on the evolving node set ΔV_t

- Run static node2vec on the initial graph G_1 to get embedding vectors Z_1
- For each subsequent timestamp $t = 2$ to T :
 - a) Find evolving node set ΔV_t
 - b) Sample new random walks only for ΔV_t
 - c) Train Skip-gram using new walks, initialized with Skip-gram t-1
 - d) Obtain embedding vectors Z_t

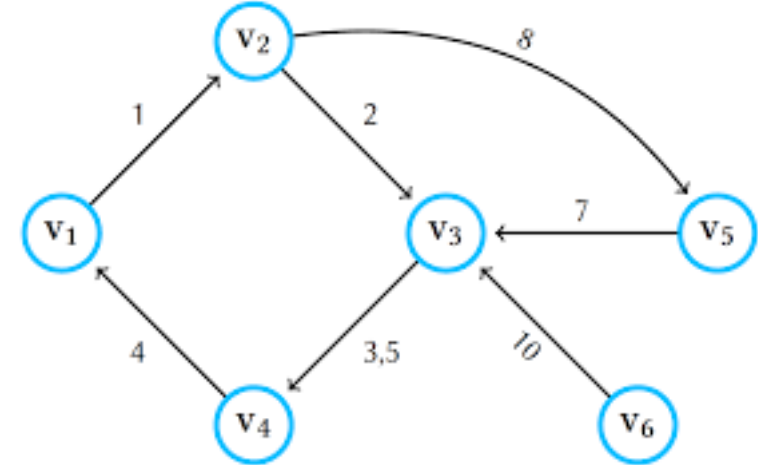
Algorithm 1 :Algorithm: Dynnode2vec

```
1: Input: Graphs  $G = G_1, G_2, \dots, G_T$ 
2: Output: Embedding vectors  $Z_1, Z_2, \dots, Z_T$ 
3: Run static node2vec for the Graph  $G_1$ 
4: for  $t = 2$  to  $N$  do
5:   Find a set of evolving nodes,  $\Delta V_t$ ,
6:   Sample new random walks ( $Walk_n$ ) for  $\Delta V_t$ 
7:   Train Skip-Gram  $Skip_t$  with  $Walk_n$  and obtain  $Z_t$ 
8: end for
```

- **Motivation:** Traditional methods often treat dynamic network as a sequence of static snapshots, which loss important temporal information
- **Solution:** Treat network as a continuous-time dynamic network, capturing the exact times of interaction



- A continuous-time dynamic network is defined as $G = (V, E_T, \mathfrak{T})$
 - V : set of nodes
 - $E_T \subseteq V \times V \times \mathbb{R}^+$: set of temporal edges between vertices in V
 - $\mathfrak{T}: E \rightarrow \mathbb{R}^+$: a function that maps each edge to a corresponding timestamp
- Temporal walk:
 - A temporal walk from v_1 to v_k in G is a sequence of nodes $\langle v_1, v_2, \dots, v_k \rangle$ such that
 - $\langle v_i, v_{i+1} \rangle$ for $1 \leq i < k$ and
 - $\mathfrak{T}(v_i, v_{i+1}) \leq \mathfrak{T}(v_{i+1}, v_{i+2})$ for $1 \leq i < (k - 1)$



➤ **Temporal random walk:**

➤ The set of temporal neighbors of a node v at time t :

$$\Gamma_t(v) = \{(w, t') \mid e = (v, w, t') \in E_T \wedge \mathcal{T}(e) > t\}$$

➤ Unbiased selection: each temporal neighbor w of node v at time t is selected

$$\Pr(w) = 1/|\Gamma_t(v)|$$

➤ Biased selection: sampling the next node in a temporal walk via temporally weighted distribution based on

$$\Pr(w) = \frac{\exp[\tau(w) - \tau(v)]}{\sum_{w' \in \Gamma_t(v)} \exp[\tau(w') - \tau(v)]}$$

Exponential decay: selecting a neighbor decreases exponentially with time

$$\Pr(w) = \frac{\delta(w)}{\sum_{w' \in \Gamma_t(v)} \delta(w')}$$

Linear decay: sorts temporal neighbors in descending order time-wise.

- Temporal context windows: To handle the temporal nature of walks - a walk to run out of temporally valid edges to traverse
 - A walk must have a minimum length ω and can extend up to a maximum length L
 - The number of context windows is defined

$$\beta = \sum_{i=1}^k |\mathcal{S}_{t_i}| - \omega + 1$$

- Learning time-preserving embeddings: maximize the likelihood of observing temporal context windows given the embeddings

$$\max_f \log \Pr(W_T = \{v_{i-\omega}, \dots, v_{i+\omega}\} \setminus v_i \mid f(v_i))$$

➡ Utilizing stochastic gradient descent

Algorithm 1 Continuous-Time Dynamic Network Embeddings

Input:

a (un)weighted and (un)directed dynamic network $G = (V, E_T, \mathcal{T})$,
temporal context window count β , context window size ω ,
embedding dimensions D ,

```

1 Set maximum walk length  $L = 80$ 
2 Initialize set of temporal walks  $\mathcal{S}_T$  to  $\emptyset$ 
3 Initialize number of context windows  $C = 0$ 
4 Precompute sampling distribution  $\mathbb{F}_s$  using  $G$ 
    $\mathbb{F}_s \in \{\text{Uniform, Exponential, Linear}\}$ 
5  $G' = (V, E_T, \mathcal{T}, \mathbb{F}_s)$ 
6 while  $\beta - C > 0$  do
7   Sample an edge  $e_* = (v, u)$  via distribution  $\mathbb{F}_s$ 
8    $t = \mathcal{T}(e_*)$ 
9    $S_t = \text{TEMPORALWALK}(G', e_* = (v, u), t, L, \omega + \beta - C - 1)$ 
10  if  $|S_t| > \omega$  then
11    Add the temporal walk  $S_t$  to  $\mathcal{S}_T$ 
12     $C = C + (|S_t| - \omega + 1)$ 
13 end while
14  $Z = \text{STOCHASTICGRADIENTDESCENT}(\omega, D, \mathcal{S}_T)$ 
15 return the dynamic node embedding matrix  $Z$ 
    
```

Algorithm 2 Temporal Random Walk

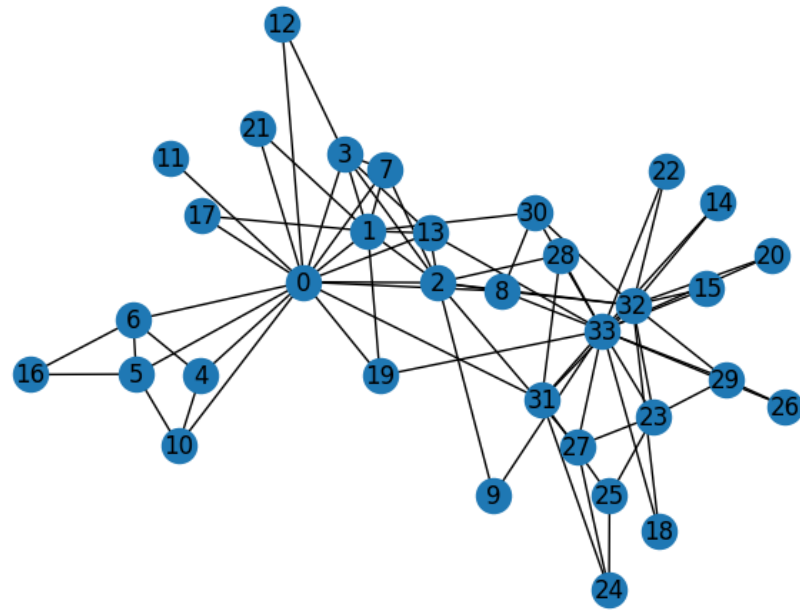
```

1 procedure TEMPORALWALK( $G', e = (s, r), t, L, C$ )
2   Initialize temporal walk  $S_t = [s, r]$ 
3   Set  $i = r$  ▷ current node
4   for  $p = 1$  to  $\min(L, C) - 1$  do
5      $\Gamma_t(i) = \{(w, t') \mid e = (i, w, t') \in E_T \wedge \mathcal{T}(i) > t\}$ 
6     if  $|\Gamma_t(i)| > 0$  then
7       Select node  $j$  from distribution  $\mathbb{F}_T(\Gamma_t(i))$ 
8       Append  $j$  to  $S_t$ 
9       Set  $t = \mathcal{T}(i, j)$ 
10      Set  $i = j$ 
11    else terminate temporal walk
12  return temporal walk  $S_t$  of length  $|S_t|$  rooted at node  $s$ 
    
```

- Initialize parameters and precompute sampling distributions
- Sample temporal walks using the specified distributions
- Use stochastic gradient descent to optimize the embeddings

```
karate_graph = nx.karate_club_graph()
# Convert the Karate Club graph to an edgelist
edgelist = nx.to_edgelist(karate_graph)
# Print the edgelist
print(edgelist)
# Create a list of edges with timestamps
edges_with_timestamps = []
for edge in edgelist:
    edges_with_timestamps.append((edge[0], edge[1], datetime.datetime.now()))
# Define the time window
time_window = datetime.timedelta(days=1)
# Perform temporal random walks
num_walks = 6
for node in karate_graph.nodes():
    for i in range(num_walks):
        walk = [node]
        current_time = datetime.datetime.now()
        while current_time < datetime.datetime.now() + time_window:
            next_hop_candidates = []
            for edge in edges_with_timestamps:
                if edge[0] == walk[-1] and edge[2] <= current_time:
                    next_hop_candidates.append(edge[1])
            if len(next_hop_candidates) > 0:
                next_node = random.choice(next_hop_candidates)
            else:
                break
            walk.append(next_node)
            current_time = datetime.datetime.now()
        print("Node " + str(node) + " walk " + str(i+1) + ": " + str(walk))
```

```
[(0, 1, {'weight': 4}), (0, 2, {'weight': 5}), (0, 3, {'weight': 3}), (0, 4, {'w
Node 0 walk 1: [0, 8, 32, 33]
Node 0 walk 2: [0, 10]
Node 0 walk 3: [0, 6, 16]
Node 0 walk 4: [0, 4, 10]
Node 0 walk 5: [0, 6, 16]
Node 0 walk 6: [0, 31, 33]
Node 1 walk 1: [1, 2, 9, 33]
Node 1 walk 2: [1, 2, 9, 33]
Node 1 walk 3: [1, 2, 28, 33]
Node 1 walk 4: [1, 21]
Node 1 walk 5: [1, 17]
```





네트워크 과학연구실
NETWORK SCIENCE LAB



가톨릭대학교
THE CATHOLIC UNIVERSITY OF KOREA

